Source: docs\technical\api-reference.md | Export: 11 November 2026

API reference

For developers/integrators. Baseline: inspected local source, 11 November 2026. This is an action-contract reference, not authorization to call write endpoints against Production. Complete field validation remains in the named owning modules. Examples use placeholders, not credentials or real order data.

Entry points and transport

Entry	Methods	Contract
`/api/catalog`	GET, POST	Public catalog/projection queries; JSON action-based operations
`/api/payments/{action}`	GET for capabilities/status; POST for other supported actions	Customer payments, signed webhook, authenticated worker and Admin reconciliation
`/sitemap.xml`	GET	Vercel rewrite to `/api/catalog?seo=sitemap`

Catalog POSTs use `Content-Type: application/json` and a body with `action`. Except anonymous enquiry submission, they require `Authorization: Bearer <Firebase ID token>`. The server obtains UID/permissions from the verified token; a body UID is not an authorization substitute. Responses are private/no-store. Catalog bodies are bounded to 100,000 bytes, with a narrower enquiry limit.

Public catalog reads

Query	Purpose
Default listing with supported cursor	Published product listing
`slug=<public-product-slug>`	Public product detail
`browse=1` plus supported discovery fields	Search/filter/sort/pagination; validate fields in `server/browse.ts`
`campaigns=1`	Public campaign list
`campaign=<slug>`	Public campaign detail
`browse=1` with supported campaign context	Campaign product discovery
`content=store`	Safe store-content/contact/theme projection
`appearance=login`	Public login imagery projection
`coupons=available`	Publicly discoverable offers without customer eligibility proof
`seo=sitemap`	XML sitemap

Unsupported combinations/fields are rejected. Do not treat a public projection as the raw private settings record. Collection visibility and product publication still apply.

Catalog POST action groups

Group	Principal actions	Access/source
Shopping	`shopping:get`, `bag:add`, `bag:adjust`, `bag:remove`, `wishlist:set`	Signed-in owner; `server/shopping.ts`
Addresses	`addresses:list`, `addresses:save`, `addresses:delete`, `addresses:default`	Signed-in owner; `server/delivery.ts`

Group	Principal actions	Access/source
Checkout/orders	`orders:review`, `orders:place`, `orders:list`, `orders:get`, `orders:request`, `orders:recover`	Signed-in owner; `server/orders.ts`
Offers	`coupons:available`	Signed-in eligibility against selected checkout source; `server/couponDiscovery.ts`
Customer identity	`profile:update`, `profile:syncDirectory`	Signed-in owner; `server/business.ts`, `server/customerDirectory.ts`
Loyalty	`loyalty:account`, `loyalty:customerHistory`; Admin account/history variants	Owner or dual-authorized Admin; `server/loyalty.ts`
Products/media	`list`, `get`, `save`, `delete`, `sign`, `verify`	Dual-authorized Admin; `server/catalog.ts`
Categories	`categories:save`, `categories:sign`, `categories:verify`	Dual-authorized Admin; category service/media workflow
Order operations	`adminOrders:list`, `adminOrders:get`, `adminOrders:update`	Dual-authorized Admin; `server/orders.ts`
Inventory/business	`inventory:adjust`, `inventory:history`, `inventory:low`, `reports:get`, `dashboard:*`, `notifications:*`	Dual-authorized Admin; `server/business.ts` and routed order actions
Customers	`customers:list`, `customers:get`, `customers:stats`, `customers:orders`, `customers:addresses`, `customers:export`	Dual-authorized Admin; directory/order services
Configuration	`settings:get`, `settings:save`, supported shipping/appearance actions	Dual-authorized Admin; `server/storeSettings.ts`, `server/shipping.ts`, `server/loginAppearance.ts`
Content	Supported `website:*`, `content:*`, `banners:*`, `campaigns:*`, `about:*`, `contact:*`	Dual-authorized Admin writes; corresponding `server/` services
Enquiries	`enquiries:submit`; Admin list/get/update/delete	Anonymous validated/rate-limited submission; dual-authorized administration

This is a service map, not a promise that every prefix accepts arbitrary suffixes. Exact allowlists, payload keys and principal checks are in `server/catalog.ts` and the owning service. UI reachability is defined by `src/admin/AdminPanel.tsx`/`src/App.tsx`; an internal reports action does not create a standalone reports screen.

Checkout review and placement

`orders:review` requires `addressId` (or null) and `couponCode` (or null), with optional `source`, `points` and `deliveryQuoteId`. `source` follows `src/checkout/source.ts`: Bag or a validated Buy Now item. The response includes items/issues, address, subtotal/discounts, loyalty, shipping, COD availability, total and fingerprint.

Shipping can be blocked/unavailable even when product/address review is valid. A ready saved flat shipping calculation authorizes shipping while distance delivery is disabled; no route quote is requested or validated. A review is not an inventory/coupon/points reservation.

`orders:place` requires `addressId`, `couponCode`, `fingerprint`, `operationId`, and supports the same source/points plus the quote ID. Amounts must not be submitted as authority. A changed confirmation can return `changed: true` with a new review; the client must show it rather than silently accept. Distance quote fields are ignored while disabled; they cannot change the flat amount. Invalid/expired/foreign quotes reject only in the explicitly enabled distance branch. Accepted-operation recovery returns the original order without a second mutation.

`settings:save` uses expected revisions and a stable operation ID for changed sections. Dispatch/COD and shipping revisions invalidate review without exposing private origin configuration. Address saves likewise use expected revision and operation identity.

Payment actions

Action	Method/access	Result/purpose
`capabilities`	GET; signed-in customer	Online availability/mode from server safeguards
`create-order`	POST; signed-in eligible customer	Validates review and quote; reserves existing order; creates/binds provider order
`verify`	POST; owning customer	Validates callback signature and reconciles trusted provider evidence
`status`	GET; owning customer	Current authoritative payment projection/recovery lookup

Action	Method/access	Result/purpose
`resume`	POST; owning customer	Existing-attempt recovery under its original mode/key
`reconcile`	POST; dual-authorized Admin	Trusted provider check and audit
`webhook`	POST; Razorpay signature/event ID	Accepts supported signed event evidence; normally 202 after acceptance
`worker`	POST; separate worker bearer secret	Processes due matching-mode/key events/attempts; writes heartbeat

Exact customer request fields are allowlisted in `server/payments/service.ts`; clients use `src/payments/client.ts`. All payment HTTP operations reject Vercel Preview. Hosted calls require HTTPS, and customer/Admin calls enforce supported origin restrictions. Do not reuse a Firebase customer token as a worker credential.

Webhooks require the original raw body, `X-Razorpay-Signature` and `X-Razorpay-Event-Id`. Supported events: `payment.authorized`, `payment.captured`, `payment.failed`, `order.paid`, `refund.created`, `refund.processed`, `refund.failed`. Signature acceptance is not completed settlement. Duplicate event identity with changed bytes rejects.

Errors, pagination and retries

Catalog errors expose safe `code`, `message`, field details and a request ID for appropriate server failures. Payment errors map invalid/unauthorized/permission/not-found/conflict/rate-limited/configuration/unavailable to bounded HTTP errors. Do not depend on a provider's raw error text being exposed.

Cursors are opaque module-owned references. Never synthesize a cursor from another user's document or assume one action's cursor fits another. Keep the same operation ID and unchanged semantic input when recovering a mutation. A fresh operation ID can represent a second business operation. Re-read/review after conflicts; distinguish transport failure from a confirmed rejection.

Sources: `api/catalog.ts`, `api/payments/[action].ts`, `server/handler.ts`, `server/catalog.ts`, `server/payments/http.ts`, `server/payments/service.ts` and named modules. Existing detailed payment reference and diagnostics guide are retained. Current live response sampling was not performed for documentation.